

Relatório Técnico – Protocolo SRTP (Simple Reliable Transport Protocol)

Nomes: Endrew Mateus dos Santos Soares, Pedro Fam Haggstram, Igor Haziel Ponticelli,
Lucas Cid Duarte

Data: 22/06/2026

Introdução

Este relatório apresenta o projeto, a implementação e a avaliação experimental do protocolo SRTP (Simple Reliable Transport Protocol), desenvolvido sobre UDP com mecanismos próprios de confiabilidade, ordenação, detecção de erros e controle de fluxo.

Foram implementadas e avaliadas três variantes do protocolo:

- Stop-and-Wait (SAW);
- Go-Back-N (GBN);
- Selective Repeat (SR).

Os experimentos foram realizados sob cenários controlados de latência, perda e reordenação de pacotes, permitindo comparar o comportamento prático dos diferentes mecanismos ARQ estudados na disciplina.

Implementação do Protocolo

Estrutura Geral

A implementação foi desenvolvida em Python utilizando UDP como camada de transporte subjacente. Toda a lógica de confiabilidade foi implementada na aplicação, sem depender do sistema operacional para retransmissão, ordenação ou recuperação de erros.

O protocolo implementado possui:

- Cabeçalho SRTP de 9 bytes;
- Números de sequência de 14 bits;
- CRC32 sobre cabeçalho e payload;
- Three-way handshake para estabelecimento da conexão;

- Encerramento com FIN e FIN+ACK;
- Timeout fixo de 100 ms;
- Modos Stop-and-Wait, Go-Back-N e Selective Repeat.

Todos os cenários executados apresentaram integridade total dos dados transferidos, com hash de saída idêntico ao hash do arquivo original.

Estrutura do Cabeçalho

O cabeçalho do protocolo contém os seguintes campos:

- SEQ (14 bits);
- SYN (1 bit);
- FIN (1 bit);
- ACK (14 bits);
- ACK Flag (1 bit);
- NACK (1 bit);
- Length (8 bits);
- CRC32 (32 bits).

O CRC32 é calculado sobre o cabeçalho e o payload, garantindo a detecção de erros de transmissão.

Segmentação

Os arquivos são divididos em segmentos com payload máximo de 255 bytes.

Nos experimentos automatizados foi utilizado um arquivo de 16.320 bytes, equivalente a 64 blocos completos de 255 bytes. Como o tamanho do arquivo é múltiplo exato do payload máximo, um pacote adicional com Length = 0 é enviado para indicar o término do fluxo de dados.

Handshake e Encerramento

O estabelecimento da conexão segue um three-way handshake:

1. O sender envia um pacote SYN.
2. O receiver responde com SYN+ACK.
3. O sender envia o ACK final.

O encerramento da sessão ocorre através da troca de mensagens FIN e FIN+ACK.

Metodologia Experimental

Os experimentos foram executados em ambiente Linux utilizando:

- namespaces de rede (ip netns);
- interfaces virtuais veth;
- tc netem para injeção de latência, perda e reordenação;
- tcpdump para captura de tráfego;
- scripts automatizados para execução das baterias de testes.

As métricas coletadas incluíram:

- throughput;
 - retransmissões;
 - duração da transferência;
 - integridade dos arquivos por hash;
 - estatísticas de envio e recepção.
-

Parte 1 – Stop-and-Wait

1. Análise do Throughput Teórico vs. Medido

A eficiência de um protocolo ARQ Stop-and-Wait pode ser calculada em função do tempo de ida e volta (RTT) e do tempo de transmissão do pacote. Segundo Kurose e Ross, a eficiência U é dada por:

$$U = (L/R) / (RTT + L/R)$$

Como o SRTP opera em uma rede local, a capacidade do enlace (R) é muito alta, tornando o tempo de transmissão (L/R) do payload de 255 bytes praticamente desprezível.

O gargalo principal passa a ser o RTT. Portanto, o throughput teórico aproximado é a razão entre o tamanho do pacote e o RTT.

Nos cenários base sem latência artificial ou perdas ($L0$ e $P0$), o RTT real da rede local é extremamente baixo. Consequentemente, o throughput medido nos experimentos $L0$ e $P0$ foi muito elevado, alcançando aproximadamente 2,09 MB/s e 2,11 MB/s, respectivamente.

Esses valores refletem o limite da pilha de rede do sistema operacional e da própria implementação em espaço de usuário, convergindo com o máximo teórico possível para o ambiente de testes.

2. Análise dos Cenários de Latência

Cenário	Latência	Throughput (B/s)	Retransmissões
L0	0 ms	2.096.312,46	0
L1	50 ms	4.797,00	0
L2	100 ms	2.419,86	66
L3	150 ms	1.614,90	67

A transição entre os cenários demonstra a limitação intrínseca do Stop-and-Wait.

De L0 para L1, o throughput cai drasticamente para cerca de 4,7 KB/s, pois o transmissor fica ocioso aguardando o ACK de cada pacote. Como o RTT ainda é inferior ao timeout fixo de 100 ms, não ocorrem retransmissões.

A quebra severa ocorre na transição de L1 para L2. Com o atraso em 100 ms, o RTT iguala ou excede ligeiramente o timeout configurado. Isso causa o disparo prematuro dos temporizadores, gerando 66 retransmissões espúrias.

Em L3, com 150 ms de atraso, o temporizador expira sistematicamente antes da chegada do ACK, resultando em 67 retransmissões e throughput de apenas 1,6 KB/s.

[Inserir captura Wireshark do cenário L2 ou L3]

3. Análise dos Cenários de Perda

Cenário	Perda	Throughput (B/s)	Retransmissões
P0	0%	2.117.720,06	0

P1	1%	134.063,55	1
P2	5%	38.204,29	4
P3	10%	17.604,21	9
P4	25%	7.277,03	22

A taxa de perda afeta o throughput de forma severamente decrescente.

Em um modelo Stop-and-Wait, cada pacote perdido bloqueia completamente a progressão da transmissão. O transmissor é obrigado a aguardar o timeout completo antes de realizar uma nova tentativa.

Como consequência, a vazão diminui rapidamente conforme a perda aumenta, chegando a apenas 7,2 KB/s em P4.

Um resultado importante observado foi que a latência se mostrou ainda mais destrutiva para o desempenho do Stop-and-Wait do que a própria perda de pacotes. Enquanto 25% de perda resultaram em aproximadamente 7,2 KB/s, apenas 150 ms de latência reduziram o throughput para 1,6 KB/s.

4. Análise do Comportamento do CRC32

O protocolo descarta silenciosamente pacotes que apresentem falha na verificação de integridade, sem o envio de um NACK.

Essa decisão evita que o protocolo tome decisões baseadas em informações potencialmente corrompidas, especialmente números de sequência inválidos.

Sem o campo CRC32, a camada de aplicação não conseguiria distinguir dados íntegros de dados corrompidos, podendo resultar na entrega de arquivos incorretos e na interpretação errada dos campos do cabeçalho.

[Inserir captura Wireshark demonstrando pacote com CRC inválido]

Parte 2 – Go-Back-N e Selective Repeat

1. Análise Comparativa sob Latência

A utilização de uma janela deslizante altera fundamentalmente o impacto da latência no desempenho.

No cenário L3 (150 ms), o throughput do Stop-and-Wait foi de apenas 1,6 KB/s.

Utilizando GBN ou SR com janela 16 no mesmo cenário, o throughput atingiu aproximadamente 15,4 KB/s.

Esse ganho ocorre porque o pipelining permite manter diversos pacotes simultaneamente em trânsito, reduzindo os períodos de ociosidade do transmissor.

2. Análise Comparativa sob Perda

No cenário P3 (10% de perda) com janela 16:

- GBN: 1.029 retransmissões e 44,2 KB/s;
- SR: 48 retransmissões e aproximadamente 1,18 MB/s.

Os resultados evidenciam a vantagem da retransmissão seletiva. Enquanto o GBN precisa reenviar grandes blocos de pacotes devido ao uso de ACKs cumulativos, o SR retransmite apenas os segmentos efetivamente perdidos.

3. Análise Comparativa sob Reordenação

No cenário R2 (25% de reordenação) com janela 16:

- GBN: 2.181 retransmissões e 83,5 KB/s;
- SR: 0 retransmissões e 110 KB/s.

O GBN interpreta pacotes fora de ordem como falhas, provocando retransmissões em lote.

O SR mantém os pacotes recebidos fora de ordem em buffer até que as lacunas sejam preenchidas, evitando retransmissões desnecessárias.

[Inserir capturas Wireshark dos cenários R2]

4. Análise do Impacto do Tamanho de Janela

O aumento do tamanho da janela produziu ganhos significativos sob latência.

Por exemplo, no cenário L1:

- GBN: 16.859 B/s (janela 4) para 45.760 B/s (janela 16);
- SR: 16.825 B/s (janela 4) para 45.793 B/s (janela 16).

Entretanto, em cenários com perda, o aumento da janela impactou fortemente o GBN.

No cenário P2:

- GBN: 36 retransmissões (janela 4) e 749 retransmissões (janela 16);
- SR: 6 retransmissões (janela 4) e 32 retransmissões (janela 16).

Isso demonstra que janelas maiores ampliam o throughput potencial, mas também ampliam o custo das perdas no Go-Back-N.

5. Conclusão Comparativa

Os resultados experimentais confirmaram o comportamento previsto pela teoria dos protocolos ARQ.

O Stop-and-Wait apresentou simplicidade de implementação, mas mostrou-se extremamente sensível à latência e à perda.

O Go-Back-N aumentou significativamente o throughput através do uso de janelas deslizantes, porém sofreu forte degradação em cenários de perda e reordenação.

O Selective Repeat apresentou o melhor desempenho geral, reduzindo retransmissões, mantendo alta vazão e lidando naturalmente com pacotes fora de ordem.

Portanto, o Selective Repeat mostrou-se a alternativa mais robusta entre as três implementações avaliadas.

Teste de Interoperabilidade

Durante o desenvolvimento foi necessário garantir aderência ao modelo de portas definido na especificação.

O receiver utiliza a porta base P , enquanto o sender utiliza a porta $P+1$ para handshake, transferência de dados e mensagens de controle.

Esse ajuste foi importante para garantir compatibilidade com implementações externas e evitar falhas de interoperabilidade.

Uso de Inteligência Artificial

Ferramentas Utilizadas

- ChatGPT (modelos GPT)
- Google Gemini

Como Foram Empregadas

As ferramentas foram utilizadas para suporte teórico, depuração do protocolo, análise dos resultados experimentais e organização do relatório técnico.

Principais Contribuições

Durante o desenvolvimento foi identificado um problema relacionado ao three-way handshake em cenários com perda.

Quando o ACK final era perdido, o receiver continuava retransmitindo mensagens SYN+ACK enquanto o sender já havia avançado para a transmissão de dados.

A análise auxiliada por IA ajudou a identificar que o sender deveria reconhecer mensagens SYN+ACK duplicadas e reenviar o ACK final do handshake, mesmo após o estabelecimento da conexão.

A correção eliminou falhas observadas em cenários com perda e aumentou a robustez da implementação.

Outras Fontes

Kurose, J. F.; Ross, K. W. Redes de Computadores e a Internet.